Static testing

October 6th, 2025
Ana Paiva, Auri Vincenzi, José Campos

In this recitation class, we are going to explore the following tools that allow one to perform *static testing* on Java projects: Checkstyle, PMD, and SpotBugs.

⚠️ Please make sure your machine is configured properly, i.e., a bash-based shell, Java JDK 11, Apache Maven v3.9.11, and that you have the latest version of IntelliJ IDEA installed. If not, please visit the 0. Setup document.

0. Pre-exercises

1. Get the `jpacman` project from here. This version includes the project's source code and some configuration files for the static testing tools listed below.

2. Extract the downloaded file.

3. Open the project on IntelliJ IDEA.

1. Static testing

1.1 Checkstyle

"Checkstyle is a static code analysis tool used in software development for checking if Java source code is compliant with specified coding rules."

To execute the static code analysis you must run

```
None
mvn site -DskipTests
```

and open the `target/site/checkstyle.html` file in your favourite browser. How many warnings and errors are reported overall and per Java file? What family of warnings/errors?

By default, Checkstyle uses the styling rules suggested by Google, which might not be inline with our project's rules. For instance, according to the report produced by Checkstyle, most of the reported warnings are related to "indentation". Let's adapt those rules to the project under test.

1. Open the `rulesets/checkstyle-rules.xml` file.
2. Adapt the `<module name="Indentation">` group according to the project under test's indentation rules. Tip: you must first identify which indentation levels are used in the project under test and then adapt the rules.
3. Re-run `mvn site -DskipTests`.
4. Open `target/site/checkstyle.html` in your favourite browser and inspect the report. At this point, you should see fewer warnings. If not, re-do these four steps.

1.2 PMD

"PMD is an open source static source code analyzer that reports on issues found within application code. PMD includes built-in rule sets and supports the ability to write custom rules."

To execute the static code analysis you must run

```
None
mvn site -DskipTests
```

and open the `target/site/pmd.html` file in your favourite browser. How many violations are reported? What type of violations? Do you think those are true violations that need to be addressed, or are they just false positives?

In the `pom.xml` file (located in the root directory of the project), the `<rulesets>` group can define as many `<ruleset>` items as you like. Under the `rulesets/` directory (in the root directory of the project under test), there are many other rules you could instantiate PMD with. Let's augment the set of rules already defined in the `pom.xml` file with some other rules under the `rulesets/` directory. In other words,

1. Open the `pom.xml` file (located in the root directory of the project).
2. Replace the following lines

```
None
<rulesets>
  <ruleset>${basedir}/rulesets/pmd-quickstart.xml</ruleset>
</rulesets>
```

with

```
None
<rulesets>
  <ruleset>${basedir}/rulesets/pmd-bestpractices.xml</ruleset>
  <ruleset>${basedir}/rulesets/pmd-security.xml</ruleset>
  <ruleset>${basedir}/rulesets/pmd-performance.xml</ruleset>
  <ruleset>${basedir}/rulesets/pmd-multithreading.xml</ruleset>
</rulesets>
```

3. Re-run `mvn site -DskipTests`.
4. Open `target/site/pmd.html` in your favourite browser and inspect the report. At this point, you should see many more or just other violations. If not, re-do the steps described above. Once more, how many violations are reported? What type of violations? Do you think those are true violations that need to be addressed, or are they just false positives?

1.3 SpotBugs

"SpotBugs is a program which uses static analysis to look for *bugs* in Java code."

To execute the static code analysis you must run

```
None
mvn site -DskipTests
```

and open `target/site/spotbugs.html` in your favourite browser. How many *bugs* are reported? What type of categories are reported? What is the priority of each reported *bug*? Do you think those are true *bugs* that need to be addressed, or are they just false positives?

SpotBugs can also be configured for the project under test. For instance, let's assume we only want SpotBugs to report *bugs* with *High* priority/confidence.

1. Open the `pom.xml` file (located in the root directory of the project).
2. Add the following lines to the two definitions of the plugin (in the reporting group).

```
None
<configuration>
<includeFilterFile>${basedir}/rulesets/spotbugs-rules.xml</includeF
ilterFile>
</configuration>
```

The `rulesets/spotbugs-rules.xml` defines the following rule

```
None
<Match>
 <Confidence value="1"/>
</Match>
```

which means that only *bugs* with a priority/confidence of 1 (*High*) would be reported by the tool.

3. Re-run `mvn site -DskipTests`.
4. Open `target/site/spotbugs.html` in your favourite browser and inspect the report. At this point, you should see fewer *bugs*. If not, re-do the steps described above. Once more, how many *bugs* are reported? What type of categories are reported? Do you think those are true *bugs* that need to be addressed, or are they just false-positives?

Note that SpotBugs also has a GUI. To run it, execute the following command:

```
None
mvn spotbugs:gui
```

2. Exercise

1. Randomly select 5 of the many *bugs* reported by two of the three static testing tools configured above. Note that we are interested in “correct positive” (i.e., a warning that is a true *bug*) and “false positive” (warning that is not a true *bug*) *bugs*.
2. Address/Fix those 5x2 *bugs*. Note that there are essentially three ways to address/fix an issue reported by a static tool:
   - Disable specific rule(s). For example, if a rule X that is enabled by default does not apply to your project’s context, perhaps it could be disabled.
   - Adapt the rules of the tool to not report that issue. For example, if a project uses 4 spaces to indent code but the tool is, by default, configured to check whether 2 spaces are used, then, the rules should be adapted but not the source code. If a project uses a mix of 2 and 4 spaces, then you should first consistently use a number of spaces in the project and then configure the tool to check that specific number of spaces.
   - Modify the source code so that the issue / warning / violation is no longer reported.
3. (re-)Perform static testing and produce a new report.

Miscellaneous

- Guide to Configuring Maven Plug-ins
- Apache Maven Checkstyle Plugin
- Apache Maven PMD Plugin
- Using the SpotBugs Maven Plugin